

When to Show a Suggestion? Integrating Human Feedback in AI-Assisted Programming

Hussein Mozannar¹, Gagan Bansal², Adam Fourney², and Eric Horvitz²

¹Massachusetts Institute of Technology, Cambridge, USA

²Microsoft Research, Redmond, USA

Abstract

AI powered code recommendation systems, such as Copilot and CodeWhisperer, provide code suggestions inside a programmer’s environment (e.g., an IDE) with the aim of improving productivity. We pursue mechanisms for leveraging signals about programmers’ acceptance and rejection of code suggestions to guide recommendations. We harness data drawn from interactions with GitHub Copilot, a system used by millions of programmers, to develop interventions that can save time for programmers. We introduce a utility-theoretic framework to drive decisions about suggestions to display versus withhold. The approach, conditional suggestion display from human feedback (CDHF), relies on a cascade of models that provide the likelihood that recommended code will be accepted. These likelihoods are used to selectively hide suggestions, reducing both latency and programmer verification time. Using data from 535 programmers, we perform a retrospective evaluation of CDHF and show that we can avoid displaying a significant fraction of suggestions that would have been rejected. We further demonstrate the importance of incorporating the programmer’s latent unobserved state in decisions about when to display suggestions through an ablation study. Finally, we showcase how using suggestion acceptance as a reward signal for guiding the display of suggestions can lead to suggestions of reduced quality, indicating an unexpected pitfall.

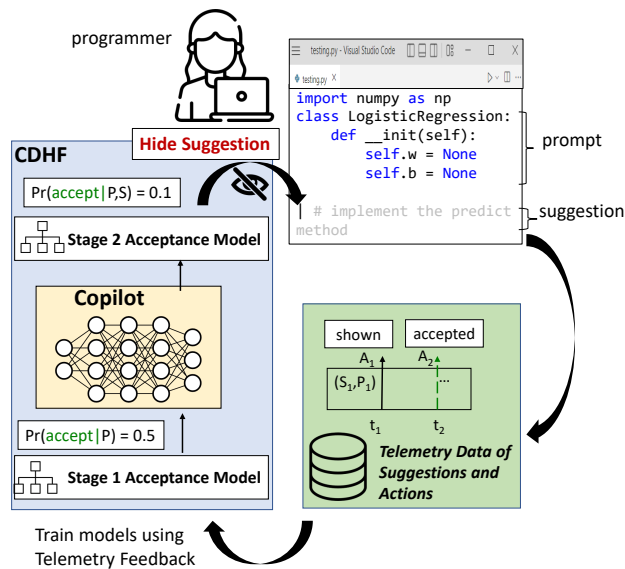


Figure 1: Operating mode of Copilot inside Visual Studio Code and how CDHF influences the interaction by selectively hiding certain suggestions. The data collected by the interaction is stored in telemetry and is used to train CDHF to create a feedback loop.

1 Introduction

Code recommendation systems powered by large-scale neural language models, such as Github Copilot [Git22] and Amazon CodeWhisperer [Ama22], are aimed at providing programmers with code suggestions to help improve their productivity. These systems usually operate by displaying the suggestion as *ghost text*—a grayed-out code suggestion inline inside the IDE. Programmers can accept the suggestion, browse through different suggestions, or reject the suggestion (see Figure 1). The code suggestions appear either at the explicit invocation of the programmer or when the programmer pauses their cursor when writing code. GitHub reported a recent randomized study with 95 participants who wrote a web server, where they found that Copilot could potentially reduce task completion time by a factor of two [Kal22]. These and other reports that the code-recommendation systems can improve programmer productivity motivate our research to pursue improvements to these systems.

Code-recommendation systems are powered by large language models (LLMs) such as GPT that are trained on standard language modeling objectives using the Common Crawl data [RWC⁺19], and then fine-tuned on public code repositories [CTJ⁺21]. The public roll-out of the code recommendation models has attracted millions of programmers, enabling a unique opportunity to leverage the data of programmers interacting with the models. In this work, we study GitHub’s Copilot which is used by millions of programmers [Git22]. For a set of programmers within our organization who consented to have their usage data collected, we collected telemetry data of Copilot suggestions, along with their associated prompts and the programmer’s action to accept or reject the suggestions. We leverage this telemetry data to design mechanisms and interventions that can improve the interaction between programmers and Copilot.

Specifically, we seek to identify *when* to show a code suggestion. We first define the expected utility of a displaying a suggestion, a value that measures the impact of showing a suggestion on the overall time to write a specific piece of code. This value provides an optimal criterion for when to show a suggestion. **However, computing the utility of suggestions is difficult and not currently feasible. Instead, we rely on the result that suggestion utility increases the more likely a suggestion is to be accepted and decreases with increasing latency to generate a suggestion—two quantities we can reliably estimate and control, respectively.** We develop a procedure, named **conditional suggestion display from human feedback (CDHF)** which guides whether to show or hide suggestions. At each pause in keystrokes, CDHF decides whether if it is worthwhile to generate a suggestion and if the programmer is likely to accept the generated suggestion. CDHF employs a cascade of models that predict acceptance of suggestions. The optimization procedure guarantees that any suggestion that was hidden (or not generated) would have been rejected if it was shown with a probability of at least p , where, e.g., p can be 0.99.

Using data from programming sessions of 535 programmers with feedback on 168k suggestions, we perform a retrospective evaluation of CDHF. We show that we can hide 25% of suggestions that were shown while guaranteeing that 95% of them would have been rejected. Further, we avoid generating 13% of these suggestions. The results show that CDHF would increase the acceptance rate by 7.2%.

The procedure allows for controlling a trade-off that balances the number of suggestions that are displayed with increases in latency, controlled with a parameter that halts generations. We note that a minimal version of CDHF has been implemented in a newer version of GitHub Copilot [Zha23] following the presentation of earlier versions of our work to GitHub. Our paper provides a roadmap for building and fielding better forms of suggestion display.

Beyond decisions about displaying recommendations, we examine the feasibility of using suggestion acceptance as a reward signal to select *which* suggestions to display and show how partial completions can be prioritized over the generations of complete code segments. While we investigate Copilot in this work, we believe our insights extend to other AI models and non-code-based tasks. Please refer to the arXiv version of this work for an appendix [MBFH23].

2 Related Work

The closest related work to ours is the procedure to selectively hide suggestions in [SDS⁺22] (quality estimation before completion, QEBC). In distinction to this work, QEBC [SDS⁺22] is not based on human feedback of accepting suggestions but rather is based on constructing a learned estimator of the quality of code completions from datasets of paired code segments and model completions. Our CDHF estimator uses real programmer behavior data and is based on data from a code-recommendation system in current use (Copilot) as opposed to custom-trained ones in [SDS⁺22]. Different metrics and datasets have been proposed to evaluate the performance of code recommendation models, but these typically assess how well the model can complete code in an offline setting without developer input rather than evaluating how well it assists programmers in situ [ZKL⁺22, LCC⁺22, EBSB22, DMN⁺22]. Integrating human preferences when training machine learning models has long been studied in the literature [KS08, MHL⁺17]. In particular, reinforcement learning from human feedback (RLHF) has been used to improve LLMs used as conversational chatbots [ZSW⁺19, BJN⁺22], notably ChatGPT [Ope22]. In contrast, CDHF uses human feedback collected organically through telemetry. The objective is fast inference to reduce latency and hiding suggestions rather than updating the LLM. Further related work can be found in the appendix. Our theoretical formulations build on earlier work on harnessing machine learning and utility to guide AI versus human-powered contributions in human-AI interactive settings [Hor99], which we apply to our setting.

3 Problem Setting

Copilot. We consider Copilot, which is a commonly used and exemplary tool of AI-powered code recommendations used by millions of programmers [Git22]. Copilot is powered by a large language model (LLM) to provide code suggestions to programmers within an IDE whenever the programmer pauses their typing. An illustration of Copilot suggesting code as an inline, single-colored snippet is displayed in Figure 1. The programmer can choose to accept this suggestion via a keyboard shortcut

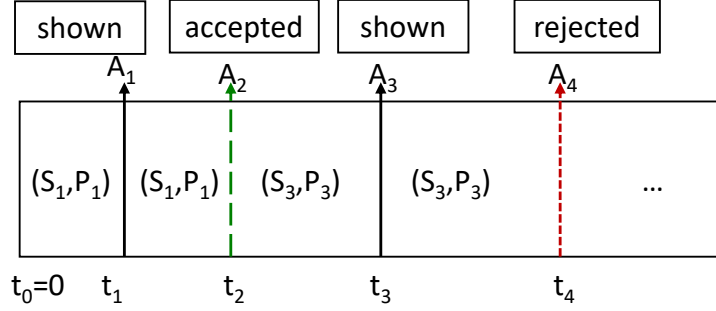


Figure 2: Schematic of telemetry with Copilot as a timeline. For a given coding session, the telemetry contains a sequence of timestamps and actions with associated prompts and suggestions.

(e.g., tab).

AI-Assisted Programming. We attempt a mathematical formalization of programming with the help of a code recommendation model such as Copilot, which we dub *AI-Assisted Programming*. The programmer wishes to complete a certain task T , for example, to implement a logistic regression classifier. As the programmer writes code starting from time 0, Copilot attempts to provide code suggestions at different times. At a given time t , Copilot¹ uses a portion of the code X_t to generate a prompt P_t , which is passed to the underlying LLM. Copilot then generates a code suggestion S_t , which is shown to the user at time $t + \tau$ where τ accounts for the LLM latency. Once the suggestion is shown, the programmer must make an action at a certain time $t' > t + \tau$, the action is $A_{t'} \in \{\text{accept, reject}\}$; the *reject* action is triggered implicitly by continuing to type.

Telemetry. Copilot logs aspects of the interactions via telemetry, which we leverage in our study. We refer to event positions drawn from a discretization of times spanning a session. Specifically, whenever a suggestion is shown, accepted or rejected, we record a tuple to the telemetry database, (t_i, A_i, P_i, S_i) , where t_i represents the within-session timestamp of the i^{th} event ($t_0 = 0$), A_i details the action taken (augmented to include ‘shown’), and P_i and S_i capture the prompt and suggestion, respectively. Figure 2 displays a portion of timeline built from telemetry data drawn from a coding session. Telemetry data from each programmer is stored in a database $D = \{(t_i, A_i, P_i, S_i)\}_{i=1}^n$ and represents a discretized representation of the interaction and provides the human feedback data we leverage.

Programmer State. When faced with a suggestion, is a programmer looking and verifying it, or rather engaged in other activities such as thinking about their code or looking at documentation? The state of the programmer is important in the expected value of the recommendation. However, we cannot answer this question as the telemetry does not capture the programmer’s activities and thinking between two consecutive time stamps t_i and t_{i+1} , i.e., the space in between the arrows in Figure 2 which we refer to as the *programmer’s latent state*. In an earlier publication [MBFH22], we describe a study of 21 participants focused on gaining an understanding of sequences of states visited during the writing of code, including latent states. The work employed videos and interviews

¹We discuss implementation details of Copilot at a high level; our work is based on the August 2022 version of Copilot.

to acquire information about the latent states. We showed in the work that including information about latent states can significantly boost predictions about accepting recommendations, motivating the collection of data beyond that captured in telemetry.

In this work, we endeavor to understand the impact of the programmer latent state denoted as ϕ_t and its effect on our ability to leverage the telemetry (human feedback data) to improve AI-code recommendation systems.

4 Theoretical Formulation of Suggestion Utility

A critical design question in programmer-Copilot interaction is **when** should the model inject a suggestion into the IDE? The version of that we Copilot provides a suggestion when it detects a brief pause in the IDE. Alternative interaction designs would require the programmer to ask for suggestions using a keyboard shortcut or to enable a mix of human and machine initiatives. Requiring the programmer to ask may lead to sub-optimal interactions because its success would rely on programmers having an accurate mental model of Copilot abilities [SGN⁺22] which can require long-term interactions with the model [BNK⁺19] or training [MSS22]. Second, requiring an explicit invocation can disrupt the natural flow of programming, breaking a state of flow achieved during intensive focus [CL14]. Designs requiring user initiative as well as those automatically displaying content can burden users with interruptions that decrease task performance [BKC01, CCH01]. We note that such costs can be inferred and accounted for formally in utility-theoretic systems [HA03, HJH99].

Ideally, Copilot should display suggestions when the suggestions provide net value to programmers. For example, consider the task of completing a function and the time taken to complete it as a proxy for the total effort. If the expected time required to verify and edit Copilot’s suggestion exceeds the time to write the code by themselves (counterfactual cost), then Copilot should not show its suggestions. Conversely, if the expected time to write exceeds the time to verify and edit, it may be useful to display the suggestion. We now formalize this intuition with a utility-theoretic formulation and, in the next section, discuss the methodology to make it practical.

Programmer Model. At a given time instance time during a session, Copilot extracts a prompt P from the code file X and generates a code suggestion S . If this suggestion is shown, we assume the programmer spends an expected time $\mathbb{E}[\text{verification}|X, S, \phi]$ to verify it and accepts the suggestion with probability $\mathbb{P}(A = \text{accept}|X, S, \phi)$. Once a suggestion is accepted, the programmer may further edit the suggestion with expected time $\mathbb{E}[\text{editing}|X, S, \phi, A = \text{accept}]$ to achieve their task. On the other hand, if the programmer rejects the suggestion, they would have to spend time writing code that achieves their task, denoted by $\mathbb{E}[\text{writing}|X, S, A = \text{reject}]$. Thus, the total time incurred with showing a suggestion, denoted as $\mathbb{E}[S \text{ shown}|X, S, \phi]$, is:

$$\begin{aligned} \mathbb{E}[S \text{ shown}|X, \phi] &= \mathbb{E}[\text{verification}|X, S, \phi] \\ &+ \mathbb{P}(A = \text{accept}|X, S, \phi) \cdot \mathbb{E}[\text{editing}|X, S, \phi, A = \text{accept}] \end{aligned} \tag{1}$$

$$+ \mathbb{P}(A = \text{reject}|X, S, \phi) \cdot \mathbb{E}[\text{writing}|X, S, \phi, A = \text{reject}]$$

While editing and writing, Copilot may further make more suggestions; thus, the editing time and writing time should include interactions with future suggestions. Now, on the other hand, if the suggestion is *not* shown, the programmer will spend time $\mathbb{E}[\text{writing}|X]$ writing code for their task. We also need to factor in latency, the time cost τ to compute a suggestion once we decide to create a suggestion. Latency is only experienced by the programmer if their latent state ϕ includes expecting a suggestion and waiting for it. If the programmer is expecting a suggestion, we should add τ to the total time when we show a suggestion; otherwise, the programmer continues to write code not expecting a suggestion.

We now define suggestion utility, a value that indicates the change in programmers' coding time due to showing the suggestion.

Definition 1 (Suggestion Utility). *The time impact δ , denoted as the suggestion utility, of showing S versus not showing is defined as:*

$$\delta = \underbrace{\mathbb{E}[\text{writing}|X, \phi]}_{S \text{ not shown}} - \underbrace{\mathbb{E}[S \text{ shown}|X, \phi]}_{S \text{ shown}} - \underbrace{\mathbb{E}[\tau|X, \phi]}_{\text{latency}} \quad (2)$$

From the above, a suggestion S at a given time should **only be shown** if $\delta > 0$ (Equation 2), where the programmer will spend less time to achieve their task if it is shown. An optimal scheme to know when to show suggestions would be to generate suggestions as frequently as possible, compute their *suggestion utility* δ , and display them if $\delta > 0$.

Feasibility of Estimating δ . Per Equation (1), computing *suggestion utility* requires the computation of four quantities: (1) the expected time spent verifying a suggestion, (2) the expected time editing a suggestion, (3) the expected time to write a segment of code and (4) the probability of accepting a suggestion. One can attempt to build an estimator for (1), by predicting from the prompt and suggestion the time spent verifying a suggestion i which would be $t_{i+1} - t_i$ using standard regression estimators. Unfortunately, using the same features and the dataset detailed in our experimental section, our best estimator is only able to achieve an $R^2 = 0.13$, which is not much better than a naive median time estimate. This may be due to the high variance and unobserved confounders governing verification time. **Estimating editing and verification time (quantities 2 and 3 above) is only more complex and challenging.** Thus, we restrict our methodology to seeing when we can evaluate δ using only our estimator for the probability of acceptance (4).

Learning Programmer's Acceptance Decisions. The full conditional for the probability that the programmer accepts a suggestion is $\mathbb{P}(A = \text{accept}|X, S, \phi)$. Given the telemetry, we can only compute $\mathbb{P}(A = \text{accept}|X, S)$ where the programmer's **latent state** cannot be observed. Using standard calibrated classification methods, we can estimate the probability $\mathbb{P}(A = \text{accept}|X, S)$ by using the actions A_i as the labels. We show that a simple mechanism of thresholding the estimated probability that the programmer accepts a suggestion is equivalent under certain assumptions to

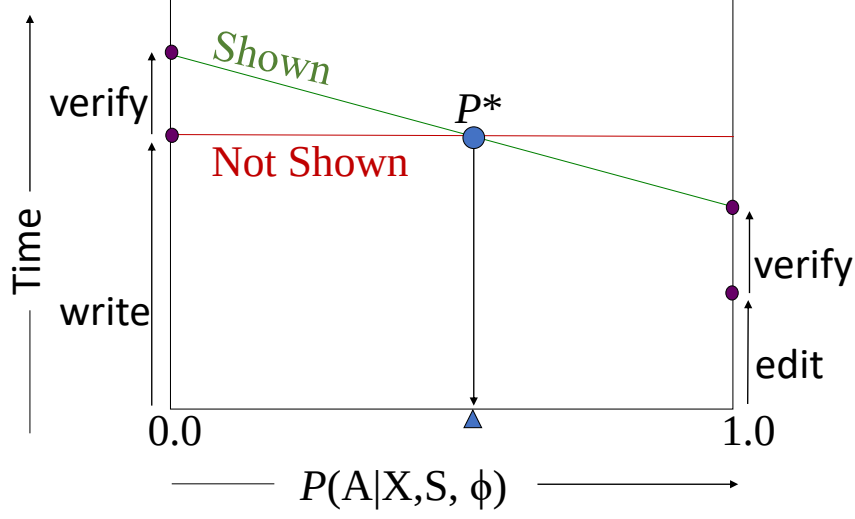


Figure 3: Graphical depiction of analysis of Proposition 1 when the latency is zero. The y-axis shows total time and the x-axis is the programmer’s probability of accepting $\mathbb{P}(A = \text{accept}|X, S, \phi)$. At probability $\mathbb{P}^*$, showing and not showing the suggestion have equal time cost.

checking if $\delta < 0$:

Proposition 1. *Under assumptions that the programmer spends more time writing code when they reject a suggestion compared to when they accept a suggestion and edit it, given specific code, suggestion, and latent state (X, S, ϕ) , if the programmer’s probability of accepting $\mathbb{P}(A = \text{accept}|X, S, \phi)$ a suggestion is below $\mathbb{P}^*$, which is defined as:*

$$\mathbb{P}^* = \frac{\mathbb{E}[\text{verification}] + \mathbb{E}[\text{latency}]}{\mathbb{E}[\text{writing}|A = \text{reject}] - \mathbb{E}[\text{editing}|A = \text{accept}]} \quad (3)$$

then the suggestion should not be shown. Note that $\mathbb{P}^$ is defined as a function $\mathbb{P}^*(X, S, \phi)$ evaluated pointwise.*

The formal statement and proof are available in the appendix. The above proposition shows that comparing the probability of acceptance to $\mathbb{P}^*$ can guide when to show the suggestion. We provide a graphical view of the analysis in Figure 3, in the spirit of related analyses on utility-guided interactive interfaces [Hor99]. Practically, if we compare the probability of acceptance to a constant lower bound of $\mathbb{P}^*$, we can guarantee that we hide suggestions only when $\delta < 0$.

Effect of Programmer Latent State. As mentioned previously, the programmer’s latent state is not available via telemetry. Thus, we can only provide predictions of $\mathbb{P}(A = \text{accept}|X, S)$ versus explicit consideration of the latent state, $\mathbb{P}(A = \text{accept}|X, S, \phi)$. In earlier work [MBFH22], we collected telemetry data of 21 programmers performing various tasks and had participants retrospectively label the telemetry with their latent state from a set of twelve unique states (1096 suggestions). We use this data to build predictive models with and without the latent state using the same methodology in the experiments section. Using a leave-one-out programmer evaluation strategy, the model without the latent state achieves accuracy 61.9 ± 1.9 while the model with the

latent state achieves 83.6 ± 2.4 , a statistically significant difference according to a paired t-test ($p = 6.9e - 7, t = 7.11$); a similar result occurs when comparing areas under the receiver operating characteristic curve (AUC). These results highlights an opportunity to gather external data beyond telemetry to build such predictive models and indicates that acceptance may not simply be a property of suggestions and code context.

5 Conditional Suggestion Display From Human Feedback

In this section, we describe the CDHF method that can be implemented using telemetry data to identify when to show suggestions, as illustrated in Figure 1. We note from Equation (3) that the higher the probability of accepting the suggestion and the lower the latency to generate the suggestion, the more likely the suggestion is useful ($\delta > 0$). Our proposed approach is as follows: Each time the programmer pauses typing, we decide using a predictor whether to show a suggestion. Crucially, we do this using a two-stage scheme to avoid generating suggestions when we know the programmer would reject them.

Display Decision. Let $m(X, S)$ be a binary predictor that denotes whether, at a given moment in the code X , we should show the suggestion S ; we call this the display decision. If $m(X, S) = 1$, we display the suggestion; otherwise, we do not. The most straightforward way to build such a function m is to estimate the programmer’s probability of accepting the suggestion: $\mathbb{P}(A = \text{accept})|X, S$ and then threshold the probability so that suggestions that fall below a probability t are hidden. However, this will lead us to generate suggestions including those that will never be shown, thus wasting computing resources. We propose to decompose the function m so that we first decide using only the code whether we can make the display decision without generating the suggestion S with a function $r(X)$. If $r(X) = 1$, we make the display decision using a stage 1 model $m_1(X)$ without generating the suggestion, otherwise if $r(X) = 0$ we generate the suggestion S and make the display decision with a stage 2 model $m_2(X, S)$ as follows:

$$m(X, S) = r(X) \cdot m_1(X) + (1 - r(X)) \cdot m_2(X, S) \quad (4)$$

This formulation allows us to avoid generating suggestions when we can make an accurate display decision in advance of knowing the suggestion. For example, in a setting where the programmer has rejected the last 30 suggestions, they are unlikely to accept the next suggestion.

Objective and Guarantees. Our objective in learning the functions r, m_1, m_2 is to (1) hide as many suggestions that would have been rejected and (2) maximize the number of display decisions made without generating the suggestion to reduce latency on the system. There is an inherent trade-off between these two objectives as making decisions with access to the suggestions would be more accurate. Moreover, we want to make sure we do not hide suggestions that would have been accepted, as this would limit the usefulness of the code assistant. Therefore, we impose a constraint that, whenever we hide a suggestion, there is at least a probability p it would have been rejected, a constraint on the true negative rate (TNR). We translate the objectives and the constraint into the

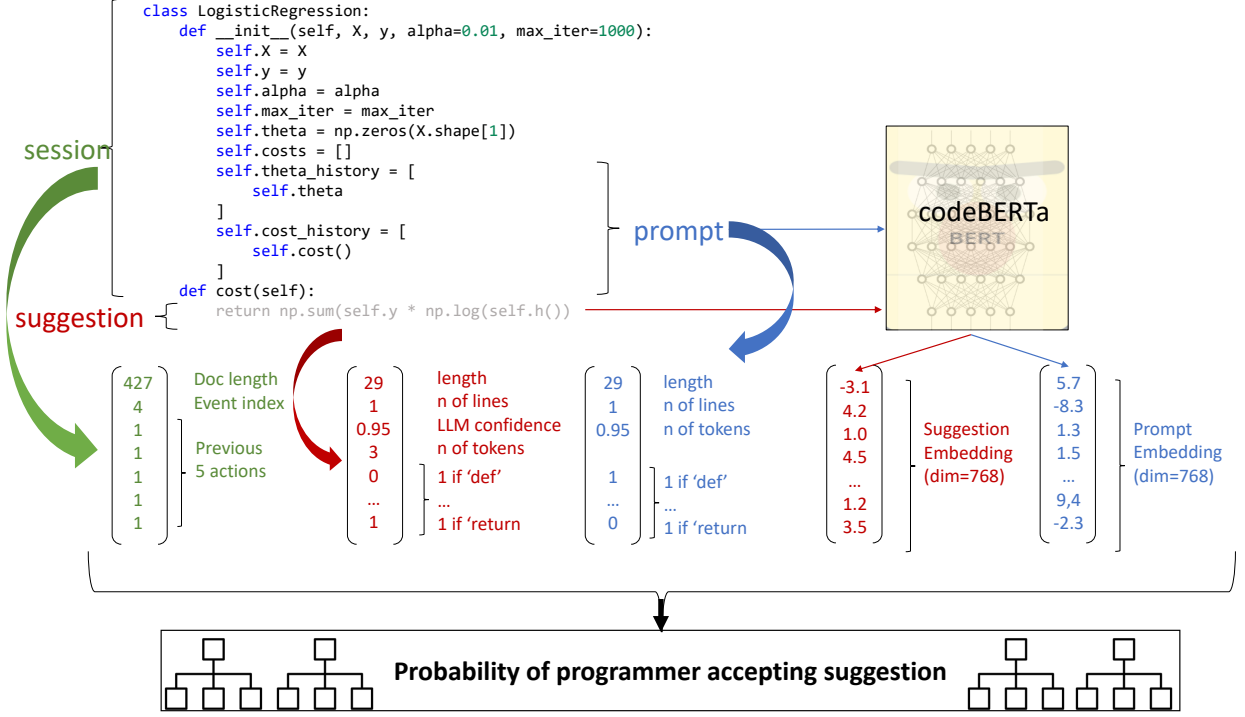


Figure 4: Features used to build action prediction model in Experiments 6, including from the suggestion, prompt, and session.

following optimization problem:

$$\max_{r, m_1, m_2} \lambda \mathbb{E}[1 - m(X, S)] + (1 - \lambda) \mathbb{E}[r(x)] \quad (5)$$

$$s.t. \quad \mathbb{P}(A = \text{reject} | m(X, S) = 0) \geq p \quad (6)$$

Parameterization. We can control the trade-off between the two objectives with a hyperparameter $\lambda \in [0, 1]$. Equivalently, instead of controlling the trade-off with λ , we can set a constraint on $\mathbb{E}[r(x)] := R$ and set $\lambda = 1$. We propose an intuitive post-hoc procedure to solve the optimization problem (5): We first learn calibrated estimators of the probability of accepting suggestions $\hat{\mathbb{P}}(A = \text{accept} | X, S)$ (with suggestion) and $\hat{\mathbb{P}}(A = \text{accept} | X)$ (without suggestion). We then parameterize:

$$m_1(X) = \mathbb{I}_{\hat{\mathbb{P}}(A = \text{accept} | X) \geq t_1}, m_2(X) = \mathbb{I}_{\hat{\mathbb{P}}(A = \text{accept} | X, S) \geq t_2}$$

and $r(X) = \mathbb{I}_{H(\hat{\mathbb{P}}(A = \text{accept} | X)) \leq t_r}$ ($H(\cdot)$ is Shannon's Entropy), and optimize *jointly* over the tuple of thresholds t_1, t_2, t_r over $[0, 1]^3$. This is a fairly efficient procedure that can achieve good results. We note that this procedure saves latency indirectly by reducing the number of LLM calls across the session and across different users, and that we should still enable the user to see the suggestion with a special keyboard shortcut to override the display decisions. In the next section, we perform a retrospective evaluation of CDHF.

6 Experiments

Our main aim with experiments is to understand how well the CDHF procedure can make display decisions in a retrospective evaluation. Code is available² and additional details can be found in the appendix.

6.1 Dataset and Feature Engineering.

Dataset. To build and evaluate our methods, we extract a large number of telemetry logs from Copilot users (mostly software engineers and researchers) at Microsoft. Programmers provided consent for the use of their data, and its use was approved by Microsoft’s ethics advisory board. Specifically, for a two-week time period, we extracted all the telemetry events for 535 users who coded in Python. This totals 4,749 coding sessions, where a session is defined as a continuous sequence of user actions with at most 30 minutes between consecutive events. These sessions are from real-world usage of Copilot for daily tasks of the software engineers and researchers, the data was collected prior to the inception of our work. On average, each user contributes nine sessions, with each session lasting 21 minutes (median, 12 minutes). Sessions contain an average of 97 events (show, accept, and reject). This totals to almost 1,675 hours of coding with 168,807 shown events and 33,523 accept events, yielding an acceptance rate of 21.4% (not meant to represent Copilot’s average acceptance rate).

Model Features. The telemetry dataset D described above contains for each user a list of events in each of their coding sessions; we denote $D_{i,j}$ to be the list of events for the j ’th session of the i ’th user. The dataset $\mathbf{D} = \{D_{i,j}\}$ contains for each user i and session j , a list of events occurring in the corresponding coding session. We extract only the accept and reject events, as well as prompt and session features of the corresponding shown events. For each prompt and suggestion pair, we extract: the programmer id as one hot vector, the length of the document, the previous five actions, suggestion features (e.g., suggestion length), previous features of the last five suggestions shown, the confidence reported by Copilot, an embedding using codeBERTa [FGT⁺20] of prompt and suggestion, presence of Python keywords (e.g., `import`, `def` `try`, etc.), and the output of the Tree-sitter Parser [Kal23]. Finally, we extract features of the prompt, including its embedding, textual features, and parser outputs. Figure 4 summarizes the feature engineering. It is crucial to note that the features do not leak any information about future events and can be computed as soon as a suggestion is generated by Copilot. For the first stage model (m_1) in CDHF, suggestion features are omitted while we include all features for the second stage model (m_2). This feature engineering incorporates past actions and suggestions that the programmer has seen and allows us to use regular ML algorithms instead of time-series methods.

²https://github.com/microsoft/coderec_programming_states

6.2 Model Evaluation

Before we evaluate CDHF, we perform an evaluation of the programmer acceptance model $m_2(X, S)$. We split the telemetry dataset in a 70:10:20 split for training, validation, and testing respectively. Importantly, we do this split in two ways: (1) by randomly splitting over programmers so that no single programmer is shared across the three splits and, (2) by randomly splitting over sessions so that users in training can also be seen in testing to allow for personalization.

Results. We evaluate different standard machine learning models on this task and find that the best-performing model is eXtreme Gradient Boosting (XGB) [CHB⁺15]. When we split across users, XGB is able to achieve 81.1% (95% CI 80.7-81.6) accuracy and, more importantly, 0.780 (95% CI 0.775-0.786) AUC. In the appendix, we show metrics for different models evaluated, including deep networks [MBFH23]. The results indicate that the model is able to distinguish between suggestions that are likely to be accepted versus those likely to be rejected. The model is also well calibrated: the expected calibration error is 0.10 [NCH15].

We note a significant increase in AUC when we allow for personalization: including programmer ID as a feature and splitting across sessions, this leads to an AUC of 0.795 (95% CI 0.789-0.801), a significant increase (basis of m_2 model). When we remove suggestion features from the model, the resulting model (basis of m_1 model) achieves an AUC of 0.631 (95% CI 0.624-0.638). The time to compute the features needed for the models and performing inference on a single data point can take 10ms with a GPU and less than 1ms on a CPU when omitting embeddings, in addition to latency of sending and receiving information between server and client. In the appendix, we show results for different ablation of model features, sample complexity plots, and feature importance plots.

6.3 Retrospective Evaluation of CDHF

We train the models m_1 and m_2 using the training set per the previous subsection. We set the thresholds t_1, t_2, t_r on the validation set for CDHF and evaluate on the test set.

Results. In Figure 5, we vary the desired TNR rate (accuracy when a suggestion is hidden) and plot how many suggestions we can hide from those previously displayed while guaranteeing the desired TNR rate. We show the behavior of the CDHF method with different λ values, or, equivalently, with different constraints on how often the m_1 model (first stage) is used: $R := \mathbb{E}[r(x)]$. To illustrate what CDHF can accomplish, we can hide 25.3% of suggestions that were shown while guaranteeing that 94.7% of them would have been rejected and avoid generating 12.9% of the suggestions. If we have no concerns for latency, we can hide 52.9% of suggestions while guaranteeing that 91.3% of them would have been rejected. Figure 5 shows how we can achieve different trade-offs by selecting an operating point on any given curve. CDHF is able to satisfy the constraint of FNR on the test set with a violation of at most 0.3% i.e., a guarantee of 95% FNR on the validation set equates to 94.7-95.3% on the test set.

Counterfactual Increase in Acceptance Rate. On the test set, the acceptance rate of suggestions is 22.5%. Retrospectively, if we had used CDHF to hide 52.9% of suggestions, we could

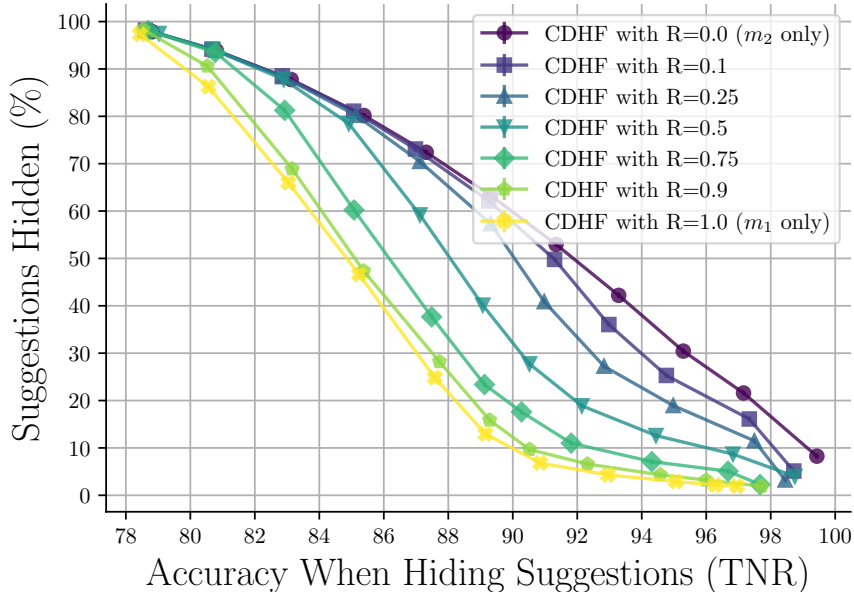


Figure 5: Evaluation of CDHF for selectively hiding suggestions. For a given constraint on FNR (accuracy when a suggestion is hidden) on the x-axis, we show on the y-axis the fraction of the total suggestions we can hide while guaranteeing the desired FNR. We plot these curves while varying how often the decision is made generating suggestions ($R := \mathbb{E}[r(x)]$, when $R=0$, we generate the suggestion then decide to hide or not, when $R=1$, we decide to hide without knowing the suggestion).

compute a counterfactual acceptance rate. The counterfactual acceptance rate can be computed as: $\frac{S_{\text{accepted}} \cdot (1 - \% \text{ hidden} \cdot (1 - TNR))}{S_{\text{shown}} \cdot \% \text{ not hidden}} = \frac{22.5(1 - 0.529 \cdot 0.087)}{0.471} = 45.6\%$, which is a 23.1 point increase, a value we expect to be an overestimate.

Discussion and Limitations. The retrospective evaluation shows that CDHF has promise in reducing developer time spent verifying suggestions or waiting for suggestions. We note that our evaluation is retrospective. Although GitHub has shown that conditional suggestion filters similar to CDHF increase the acceptance rate of suggestions, a user study is required to verify whether the method makes programmers more productive. As Goodhart’s law states, once a metric becomes a target, it ceases to become a good measure; acceptance rate is no exception. Moreover, if CDHF is not trained with sufficient data that captures the programmer’s use cases, it can make the programming experience worse by hiding useful suggestions. Moreover, a rejected suggestion may still be useful, which we do not account for here. Finally, the optimization problem in (5) is amenable to procedures inspired by learning to defer [MS20] that can outperform the post-hoc procedure proposed.

7 Which Suggestion to Show?

We focus in this study on the problem of when to display suggestions. We did not tackle the question of which suggestions to display among a candidate set. Given access to telemetry data, which consists of contextualized suggestions with accept and reject signals, one can interpret an accept as the act

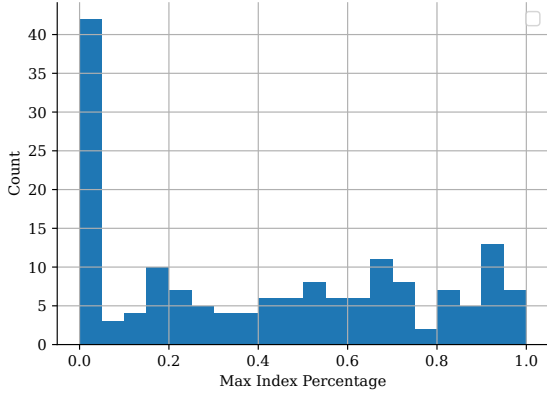
of preferring a suggestion over no suggestion. It is reasonable to harness the telemetry data as a preference dataset and build a reward model of programmers’ preferences, which would be equivalent to estimating the programmer’s acceptance probability $\hat{\mathbb{P}}(A = \text{accept}|X, S)$. Thus, a reasonable procedure is to take a candidate set of suggestions $\mathcal{S}$ and display the suggestion that maximizes the probability of acceptance across the set; this is essentially the best-of- n baseline approach in RLHF [RSM⁺23].

Potential Bias Towards Short Suggestions. We hypothesize that such a ranking scheme would not be productive and can lead to poor suggestions of short length. Our rationale is the following: suppose the LLM is able to generate a multi-line suggestion S for a user query that approximately matches what the user desires. To maximize the probability that the user accepts the suggestion, it would be advantageous to split the suggestion S line-by-line and display it to the user step-by-step. The reasoning is that it is more likely for the first line of S to be correct rather than all of S being correct, hence being more likely to be accepted.

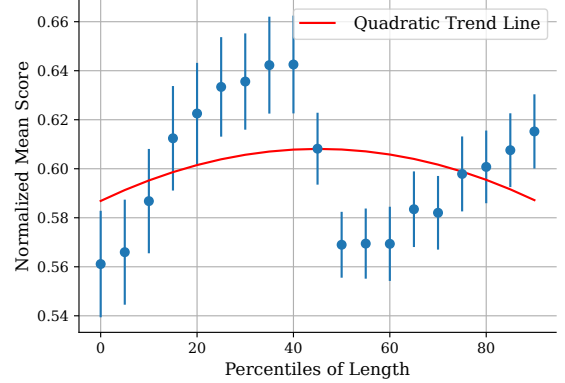
Experiment. To test this hypothesis, we perform the following experiment: We learn a model m of suggestion acceptance given only the prompt and suggestion embeddings with no session features on the telemetry data from the previous section. We then leverage the HumanEval dataset [CTJ⁺21], which consists of 164 Python problems, each with an associated docstring and a ground truth function body solution. Solutions have at least two lines and seven median lines of code. Given the model m and each problem, we let the prompt be the concatenation of the docstring and the first k lines of the solution and let the candidate set of suggestions $\mathcal{S}$ be as follows: Given the solution S represented as an array of tokens of length N , we let $\mathcal{S} = \{S[:i]\}_{i=1}^N$. For example, if the solution S was "return np.mean(x)", then $\mathcal{S} = \{"return", "return np.mean(x)"\}$.

Results. We vary the parameter k in the set $\{0, 1, 2, 3\}$ so that the prompt goes from the docstring to include lines of the solution. When $k = 0$, the normalized length of the highest-rated suggestion, according to the model across the 164 problems, is almost uniform across $[0, 1]$, a Kolmogorov–Smirnov test compared to the uniform distribution has a p-value of 0.53 (KS=0.06). Optimally, we want the normalized length to cluster around 1 to include the full solution. However, when $k > 0$, meaning that the prompt includes lines of code, we find that for over 60 of the 164 problems, the highest-scored suggestion lies in $[0, 0.2]$, and, for at least 40 problems, it is the first token. A histogram showing this phenomenon is in Figure 11 for $k = 1$. This provides some evidence that optimizing for acceptance can be biased toward shorter suggestions since the highest-ranked suggestion is often in the first few lines of the solution.

Limitations. However, there are important limitations in our experiment. First, the model m is only trained on Copilot suggestions. Thus, the bias towards short suggestions can be due in part to Copilot potentially showing short suggestions. Maximizing acceptance would not alleviate such a bias. Second, while the use of embeddings of the suggestions for the reward model led to accurate predictions of accepts (AUC=0.701), they might be biased in some ways as compared to alliance on fine-tuning the language model.



(a) Histogram of position of max suggestion



(b) Probability of acceptance by length

Figure 6: Plots for the experiment on ranking suggestions by the probability of acceptance. Histogram (a) shows in which length percentile bin the suggestion maximizing the probability of acceptance lies and Graph (b) shows the acceptance score by increasing the length of the suggestion. These plots are for $k = 1$ (docstring + first line of solution)

8 Conclusion

We proposed a strategy to decide when to display code suggestions in AI-assisted programming to improve time efficiency. This strategy was based on a utility theory formulation and employs a two-stage procedure using a predictive model of suggestion acceptance. A retrospective evaluation showed that we can reduce the number of suggestions and thus programmers’ time without sacrificing the utility of Copilot. However, a prospective study that evaluates the impact of Copilot with and without CDHF could help with conclusive evaluation and is the basis of future work. Moreover, future work will attempt to directly estimate Proposition 1 leveraging improved methods. We don’t believe that CDHF can introduce negative consequences beyond what Copilot introduces to the programmer as it functions as a filtering mechanism for unhelpful suggestions. Moreover, we believe that the CDHF methodology can be employed in a wide range of streaming human-AI collaboration tasks such as assisted writing. Future work will incorporate the latent state of the programmer into the predictive models, investigate how to rank suggestions using the models from CDHF, and validate the efficacy of CDHF in user studies.

Acknowledgments

Hussein Mozannar partly conducted this work during an internship at Microsoft Research (MSR). We acknowledge valuable feedback from colleagues across MSR and GitHub including Saleema Amershi, Victor Dibia, Forough Poursabzi, Andrew Rice, Eirini Kalliamvakou, and Edward Aftandilian.

References

- [Ama22] Amazon. ML-powered coding companion – amazon codewhisperer, 2022.
- [BCC⁺23] Umang Bhatt, Valerie Chen, Katherine M Collins, Parameswaran Kamalaruban, Emma Kallina, Adrian Weller, and Ameet Talwalkar. Learning personalized decision support policies. *arXiv preprint arXiv:2304.06701*, 2023.
- [BJN⁺22] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022.
- [BJP22] Shraddha Barke, Michael B James, and Nadia Polikarpova. Grounded copilot: How programmers interact with code-generating models. *arXiv preprint arXiv:2206.15000*, 2022.
- [BKC01] Brian P Bailey, Joseph A Konstan, and John V Carlis. The effects of interruptions on task performance, annoyance, and anxiety in the user interface. In *Interact*, volume 1, pages 593–601, 2001.
- [BMR⁺20] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [BNK⁺19] Gagan Bansal, Besmira Nushi, Ece Kamar, Walter S Lasecki, Daniel S Weld, and Eric Horvitz. Beyond accuracy: The role of mental models in human-ai team performance. In *Proceedings of HCOMP*, volume 7, pages 2–11, 2019.
- [CCH01] Edward Cutrell, Mary Czerwinski, and Eric Horvitz. Notification, disruption, and memory: Effects of messaging interruptions on memory and performance. In *IFIP TC13 International Conference on Human-Computer Interaction*, 2001.
- [CG16] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD ’16, pages 785–794, New York, NY, USA, 2016. ACM.
- [CHB⁺15] Tianqi Chen, Tong He, Michael Benesty, Vadim Khotilovich, Yuan Tang, Hyunsu Cho, Kailong Chen, et al. Xgboost: extreme gradient boosting. *R package version 0.4-2*, 1(4):1–4, 2015.
- [CL14] Mihaly Csikszentmihalyi and Reed Larson. *Flow and the foundations of positive psychology*, volume 10. Springer, 2014.

-
- [CLB⁺17] Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *Advances in neural information processing systems*, 30, 2017.
- [CTJ⁺21] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [DK18] John J Dudley and Per Ola Kristensson. A review of user interface design for interactive machine learning. *ACM Transactions on Interactive Intelligent Systems (TiiS)*, 8(2):1–37, 2018.
- [DMN⁺22] Arghavan Moradi Dakhel, Vahid Majdinasab, Amin Nikanjam, Foutse Khomh, Michel C Desmarais, Zhen Ming, et al. Github copilot ai pair programmer: Asset or liability? *arXiv preprint arXiv:2206.15331*, 2022.
- [EBSB22] Mikhail Evtikhiev, Egor Bogomolov, Yaroslav Sokolov, and Timofey Bryksin. Out of the bleu: how should we assess quality of the code generation models? *arXiv preprint arXiv:2208.03133*, 2022.
- [FGT⁺20] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155*, 2020.
- [Git22] Github. Github copilot - your ai pair programmer, 2022.
- [HA03] Eric Horvitz and Johnson Apacible. Learning and reasoning about interruption. In *Proceedings of the 5th International Conference on Multimodal Interfaces, ICMI '03*, page 20–27, 2003.
- [HBK⁺21] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, et al. Measuring coding challenge competence with apps. *arXiv preprint arXiv:2105.09938*, 2021.
- [HJH99] Eric Horvitz, Andy Jacobs, and David Hovel. Attention-sensitive alerting. In *Proceedings of UAI*, page 305–313, 1999.
- [Hor99] Eric Horvitz. Principles of mixed-initiative user interfaces. In *Proceedings of CHI*, pages 159–166, 1999.
- [Kal22] Eirini Kalliamvakou. Research: Quantifying github copilot’s impact on developer productivity and happiness, Sep 2022.
- [Kal23] Eirini Kalliamvakou. Tree-sitter parser generator tool, Jan 2023.

-
- [KS08] W Bradley Knox and Peter Stone. Tamer: Training an agent manually via evaluative reinforcement. In *2008 7th IEEE international conference on development and learning*, pages 292–297. IEEE, 2008.
- [LCC⁺22] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. Competition-level code generation with alphacode. *arXiv preprint arXiv:2203.07814*, 2022.
- [MBFH22] Hussein Mozannar, Gagan Bansal, Adam Fourney, and Eric Horvitz. Reading between the lines: Modeling user behavior and costs in ai-assisted programming. *arXiv preprint arXiv:2210.14306*, 2022.
- [MBFH23] Hussein Mozannar, Gagan Bansal, Adam Fourney, and Eric Horvitz. When to show a suggestion? integrating human feedback in ai-assisted programming. *arXiv preprint arXiv:2306.04930*, 2023.
- [MHL⁺17] James MacGlashan, Mark K Ho, Robert Loftin, Bei Peng, Guan Wang, David L Roberts, Matthew E Taylor, and Michael L Littman. Interactive learning from policy-dependent human feedback. In *ICML*, pages 2285–2294. PMLR, 2017.
- [MPZ18] David Madras, Toni Pitassi, and Richard Zemel. Predict responsibly: improving fairness and accuracy by learning to defer. *Advances in Neural Information Processing Systems*, 31, 2018.
- [MS20] Hussein Mozannar and David Sontag. Consistent estimators for learning to defer to an expert. In *ICML*, pages 7076–7087. PMLR, 2020.
- [MSS22] Hussein Mozannar, Arvind Satyanarayan, and David Sontag. Teaching humans when to defer to a classifier via exemplars. In *AAAI*, volume 36, pages 5323–5331, 2022.
- [NCH15] Mahdi Pakdaman Naeini, Gregory Cooper, and Milos Hauskrecht. Obtaining well calibrated probabilities using bayesian binning. In *Proceedings of AAAI*, 2015.
- [ODR21] Nastaran Okati, Abir De, and Manuel Rodriguez. Differentiable learning under triage. *Advances in Neural Information Processing Systems*, 34:9140–9151, 2021.
- [Ope22] OpenAI. Chatgpt: Optimizing language models for dialogue, 2022.
- [PGM⁺19] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, pages 8024–8035. Curran Associates, Inc., 2019.

-
- [PVG⁺11] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [RSM⁺23] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *arXiv preprint arXiv:2305.18290*, 2023.
- [RWC⁺19] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- [SDS⁺22] Zhensu Sun, Xiaoning Du, Fu Song, Shangwen Wang, Mingze Ni, and Li Li. Learning to prevent profitless neural code completion. *arXiv preprint arXiv:2209.05948*, 2022.
- [SGN⁺22] Advait Sarkar, Andrew D Gordon, Carina Negreanu, Christian Poelitz, Sruti Srinivasa Ragavan, and Ben Zorn. What is it like to program with artificial intelligence? *arXiv preprint arXiv:2208.06213*, 2022.
- [SK09] Xiaoyuan Su and Taghi M Khoshgoftaar. A survey of collaborative filtering techniques. *Advances in artificial intelligence*, 2009, 2009.
- [TN22] Maxim Tabachnyk Tabachnyk and Stoyan Nikolov. ML-enhanced code completion improves developer productivity, Jul 2022.
- [VZG22] Priyan Vaithilingam, Tianyi Zhang, and Elena L Glassman. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts*, pages 1–7, 2022.
- [WMH⁺21] Justin D Weisz, Michael Muller, Stephanie Houde, John Richards, Steven I Ross, Fernando Martinez, Mayank Agarwal, and Kartik Talamadupula. Perfection not required? human-ai partnerships in code translation. In *26th International Conference on Intelligent User Interfaces*, pages 402–412, 2021.
- [Zha23] Shuyin Zhao. GitHub Copilot now has a better AI model and new capabilities. <https://github.blog/2023-02-14-github-copilot-now-has-a-better-ai-model-and-new-capabilities/>, February 2023.
- [ZKL⁺22] Albert Ziegler, Eirini Kalliamvakou, X Alice Li, Andrew Rice, Devon Rifkin, Shawn Simister, Ganesh Sittampalam, and Edward Aftandilian. Productivity assessment of neural code completion. In *Proceedings of SIGPLAN*, pages 21–29, 2022.
- [ZSW⁺19] Daniel M Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593*, 2019.

A Extended Related Work

AI-Assisted Programming. Large language models (LLMs) such as GPT-3 [BMR⁺20], have been widely used in natural language processing. One example of this is Codex [CTJ⁺21], a GPT model trained on 54 million GitHub repositories, which demonstrates the effectiveness of LLMs in solving various programming tasks. For instance, Codex was tested on the HumanEval dataset of 164 programming problems, where it was asked to write the function body from a docstring and achieved 37.7% accuracy with a single generation [CTJ⁺21]. Different metrics and datasets have been proposed to evaluate the performance of code recommendation models, but these typically assess how well the model can complete code in an offline setting without developer input, rather than evaluating how well it assists programmers in-situ [HBK⁺21, LCC⁺22, EBSB22, DMN⁺22]. Researchers have found that developers do not need a perfect recommendation model for it to be useful. Weisz et al. conducted interviews with developers and found that they did not require a perfect recommendation model for the model to be useful [WMH⁺21], while Ziegler et al. surveyed over 2,000 Copilot users and found that they felt more productive using Copilot [ZKL⁺22]. A study by Google found that an internal Copilot model had a 6% reduction in 'coding iteration time' [TN22]. On the other hand, a study of 24 participants by Vaithilingam et al. showed no significant improvement in task completion time, yet participants stated a clear preference for Copilot [VZG22]. [BJP22] showed that interaction with Copilot falls into two broad categories: the programmer is either in 'acceleration mode' or in 'exploration mode'. A similar strategy to selectively hide code suggestions was proposed in [SDS⁺22] (Quality Estimation Before Completion, QEBC). However, QEBC [SDS⁺22] is not based on human feedback of accepting suggestions, but rather is based on constructing a learned estimator of the quality of code completions from datasets of paired code segments and model completions. In contrast, our CDHF estimator uses real programmer behavior data and is based on data from a code-recommendation system in current use (Copilot) as opposed to custom-trained ones in [SDS⁺22].

Human Feedback. Integrating human preference when training machine learning based models have long been studied in the literature [KS08, MHL⁺17]. In particular, Reinforcement Learning from Human Feedback is an approach where the designer first gathers explicit human preference over actions which is used to improve the model using RL [CLB⁺17]. More recently, this approach has been used to improve LLMs for different tasks [ZSW⁺19, BJN⁺22] notably including ChatGPT [Ope22] and consists of three steps: gather human preference over options, train a reward model of human preference, use the reward model to update the LLM using RL. In contrast, our approach CDHF uses implicit human feedback through telemetry more readily collected which is not fully reflective of true preference and avoids updating the LLM. Collaborative filtering employs human preference data to re-rank content [SK09], though avoids the complexities of both generating and ranking content which is required here. Our theoretical formulations in Section 4 build on the work of interactive user interfaces [Hor99, DK18] and generalize them to our setting. Work on algorithmic deferral [MPZ18, MS20, ODR21] investigates a similar question on whether the human or the AI should accomplish the task and is based on error estimation (instead of time cost estimation here)

of the human versus the AI, as well as work on personalized decision support policies [BCC⁺23] investigates whether support should be shown but not when.

B Derivation of $\mathbb{P}^*$

Proposition 1. *Under assumptions of the programmers model, notably that the programmer spends more time writing code when they reject a suggestion compared to when they accept a suggestion and edit it. Given specific code, suggestion and latent state (X, S, ϕ) , if the programmer’s probability of accepting $\mathbb{P}(A = \text{accept}|X, S, \phi)$ a suggestion is below $\mathbb{P}^*$ defined as:*

$$\mathbb{P}^* = \frac{\mathbb{E}[\text{verification}] + \mathbb{E}[\tau|X, \phi]}{\mathbb{E}[\text{writing}|A = \text{reject}] - \mathbb{E}[\text{editing}|A = \text{accept}]}$$

then the suggestion should not be shown. Note that $\mathbb{P}^$ is defined as a function $\mathbb{P}^*(X, S, \phi)$ evaluated pointwise.*

Proof. Starting from the equation $\delta = 0$, assume that the programmer has only two actions of accept or reject. This is backed by our analysis of our telemetry sample where we noticed that less than 1% of suggestions are browsed. We first have:

$$\begin{aligned} & - (\mathbb{E}[\text{verification}|X, S, \phi] + \mathbb{P}(A = \text{accept}|X, S, \phi) \cdot \mathbb{E}[\text{editing}|X, S, \phi, A = \text{accept}] \\ & + \mathbb{P}(A = \text{reject}|X, S, \phi) \cdot \mathbb{E}[\text{writing}|X, S, \phi, A = \text{reject}]) + \mathbb{E}[\text{writing}|X, \phi] - \mathbb{E}[\tau|X, \phi] = 0 \end{aligned}$$

We now replace $\mathbb{P}(A = \text{accept}|X, S, \phi)$ by $\mathbb{P}^*(X, S, \phi)$ and move around terms:

$$\begin{aligned} & \mathbb{P}^*(X, S, \phi) \cdot (-\mathbb{E}[\text{editing}|X, S, \phi, A = \text{accept}] + \mathbb{E}[\text{writing}|X, S, \phi, A = \text{reject}]) \\ & = \mathbb{E}[\text{verification}|X, S, \phi] + \mathbb{E}[\text{writing}|X, S, \phi, A = \text{reject}] - \mathbb{E}[\text{writing}|X, \phi] + \mathbb{E}[\tau|X, \phi] \end{aligned}$$

Assuming that $\mathbb{E}[\text{writing}|X, S, \phi, A = \text{reject}] - \mathbb{E}[\text{editing}|X, S, \phi, A = \text{accept}]$ is > 0 , meaning when suggestions are accepted, the editing time for them is less than the time to write the suggestions. We can then separate $\mathbb{P}^*(X, S, \phi)$ to the LHS with full equivalence (equality still holds without this assumption):

$$\mathbb{P}^*(X, S, \phi) = \frac{\mathbb{E}[\text{verification}|X, S, \phi] + \mathbb{E}[\text{writing}|X, S, \phi, A = \text{reject}] - \mathbb{E}[\text{writing}|X, \phi] + \mathbb{E}[\tau|X, \phi]}{\mathbb{E}[\text{writing}|X, S, \phi, A = \text{reject}] - \mathbb{E}[\text{editing}|X, S, \phi, A = \text{accept}]}$$

Note that $\mathbb{P}^*(X, S, \phi)$ is not necessarily a valid probability in $[0, 1]$. If we assume that if the programmer rejected a suggestion, they did not benefit at all from it when writing the code afterwards meaning $\mathbb{E}[\text{writing}|X, S, \phi, A = \text{reject}] = \mathbb{E}[\text{writing}|X, \phi]$, we arrive at:

$$\mathbb{P}^*(X, S, \phi) = \frac{\mathbb{E}[\text{verification}|X, S, \phi] + \mathbb{E}[\tau|X, \phi]}{\mathbb{E}[\text{writing}|X, S, \phi, A = \text{reject}] - \mathbb{E}[\text{editing}|X, S, \phi, A = \text{accept}]}$$

Assuming latency is zero meaning $\tau = 0$, then it is clear that $\mathbb{P}^*(X, S, \phi) = 0$ if and only if verification time is negligible: $\mathbb{E}[\text{verification}|X, S, \phi] = 0$, and by assumption it is $\mathbb{P}^*(X, S, \phi) \geq 0$ since the denominator is positive. The implication is that if $\mathbb{P}(A = \text{accept}|X, S, \phi) \leq \mathbb{P}^*(X, S, \phi)$, we should not show the suggestion which follows directly from the equation of $\delta \leq 0$, and similarly if $\mathbb{P}(A = \text{accept}|X, S, \phi) \geq \mathbb{P}^*(X, S, \phi)$ we show the suggestion.

To restate, this derivation made the following assumptions:

- The programmer has only two actions of accept or reject.
- $\mathbb{E}[\text{writing}|X, S, \phi, A = \text{reject}] - \mathbb{E}[\text{editing}|X, S, \phi, A = \text{accept}]$ is > 0 , meaning when suggestions are accepted, the editing time for them is less than the time to write the suggestions.
- If we assume that if the programmer rejected a suggestion, they did not benefit at all from it when writing the code afterwards meaning $\mathbb{E}[\text{writing}|X, S, \phi, A = \text{reject}] = \mathbb{E}[\text{writing}|X, \phi]$.

□

C Model Evaluation and Analysis

All experiments were run with Python 3.8 on a machine with a single A100 GPU.

Confidence intervals in the body are obtained by bootstrapping with 1000 bootstrap samples for accuracy and AUC.

In table 1 we evaluate the performance of different models for predicting acceptance of suggestions. We use Scikit-learn [PVG⁺11] to train the Random Forrest and logistic regression models, XGBoost library to train our XGB models [CG16] and PyTorch to train the fully connected neural network [PGM⁺19]. The FCNN is a 3 layer network with 50 hidden units trained using Adam with a learning rate of $1e-2$. We used the validation set to select the number of hidden units as well as the picking the best model after training for 100 epochs. We did not employ any hyperparameter tuning for the XGBoost models and used the standard parameters.

Table 1: Comparison of different classifiers on the test set based on AU-ROC, accuracy, and macro f1 for the full model to predict programmer suggestion acceptance. FCNN refers to a fully connected neural network.

Method	AU-ROC	Accuracy (%)	Macro F1 (%)
XGBoost	0.780	81.1	64
Logistic Regression	0.726	79.5	58
Random Forrest	0.756	80.9	61
FCNN	0.741	80.2	59
Baseline	0.5	78.9	44

In figure 7 we showcase that the stage 2 model m_2 is well calibrated.

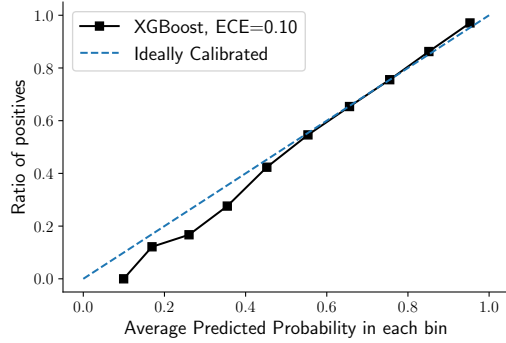


Figure 7: Calibration curve for the XGBoost stage 2 model.

Sample Complexity. To understand how many samples we might need to train individualized CDHF models, we perform a sample complexity analysis on the acceptance prediction stage 2 model where we train with a random fraction of the training dataset in Figure 8. With 1% of the training dataset which equates to 1688 training samples, performance reaches 0.69 AU-ROC. With 25% of training data, the model can achieve 0.75 AU-ROC.

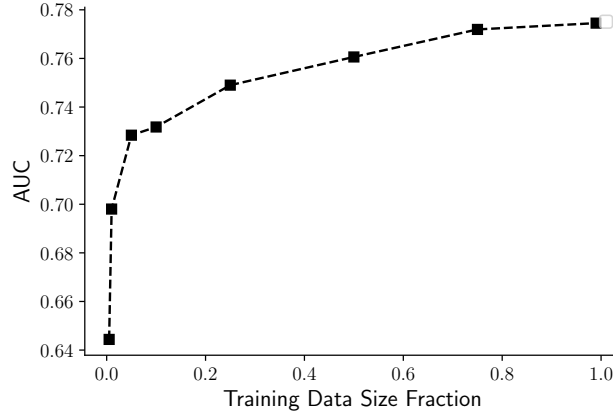


Figure 8: Sample complexity analysis of the XGBoost stage 2 model when trained on a fraction of the training data and plotting the AU-ROC on the full test set.

Factors Influencing Programming Actions. An analysis of the XGBoost stage 2 model, reveals factors that correlate with programmers’ decisions. We examine feature importance weights (Figure 9). Specifically, we report feature F-score counts, which capture how often each feature was split in any of the trees in the XGBoost model. The two most important features are the confidence of the suggestion from Copilot’s core suggestion model, and the length of the suggestion. Together, these two features yield a model that achieves an AUROC of 0.71. Other important features include the context of the current event in the coding session such as if the last suggestion was accepted or not. We also see that textual elements of the suggestion are important. For example, the feature indicating if the current suggestion includes the character ‘#’ (which is used to indicate a comment

in Python) was split a total of 8 times. These features should not be interpreted in a causal fashion but rather as being correlated with the behavior of programmers.

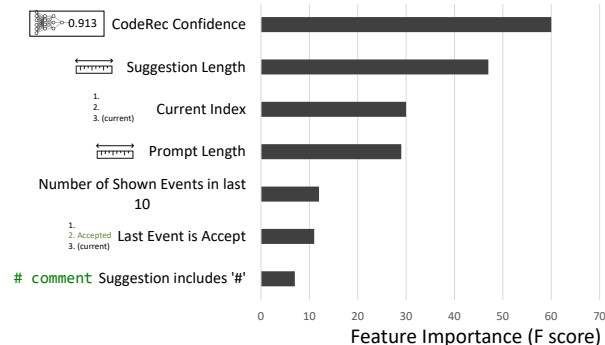
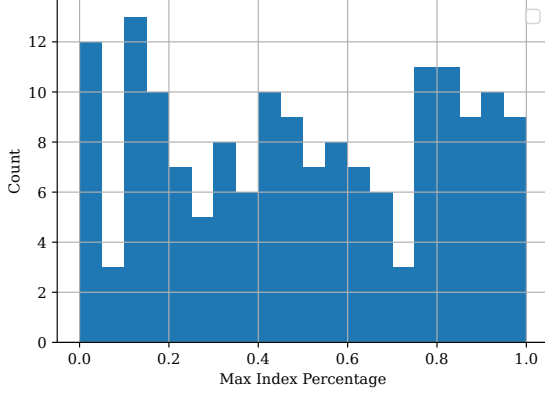


Figure 9: Feature importance for the seven highest-rated unique features of the model for predicting the likelihood of accepting a suggestion. The feature importance is in terms of the F score which counts how often a feature was split on in the tree ensemble.

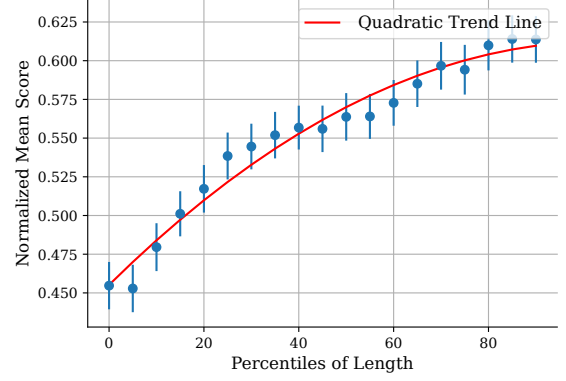
Analysis of Suggestions. The model learned on the large sample of telemetry can also be applied to the telemetry collected in the 21-participant user study of [MBFH22]. When we evaluate the model on the user study’s 1029 accepted and rejected events we obtain an AU-ROC of 0.73. Inspecting these results further, we find that there are at least two defined clusters for suggestions predicted most likely be rejected: (1) single character non-alphabetic suggestions such as (,), [, :, ;, and (2) mid-word completions such as ‘agonal()’ (completion of ‘diagonal()’), ‘lass LogisticRegression:’ (completion of ‘class Logistic Regression:’). We hypothesize that for cluster (1) the suggestions were too small to be noticed. For cluster (2), we hypothesize that the programmer was already in the act of typing, so the suggestions may have been a distraction (i.e., the interruption cost more than was saved by eliminating the physical act of typing a few already-determined keystrokes).

D Which Suggestion to Show: Plots

Following the last section of the paper, we show two plots for each $k \in \{0, 1, 2, 3\}$: 1) histogram showing in which length percentile in terms the ground truth solution does the suggestion with highest acceptance probability (according to the model) lies and 2) normalized probability of acceptance by the length of the suggestion (for each example, we normalize the raw probability of acceptance by the maximum acceptance probability across all length for the given example - we observe the same trend without normalizing).

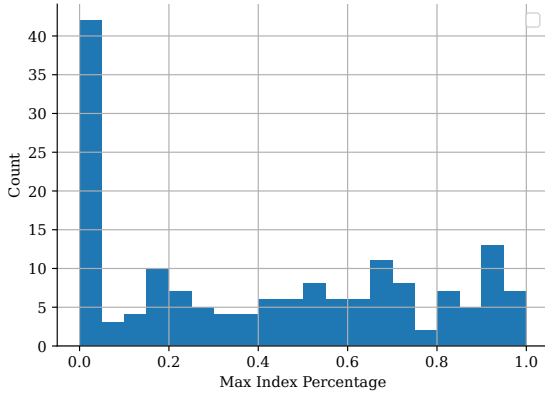


(a) Histogram of position of max suggestion

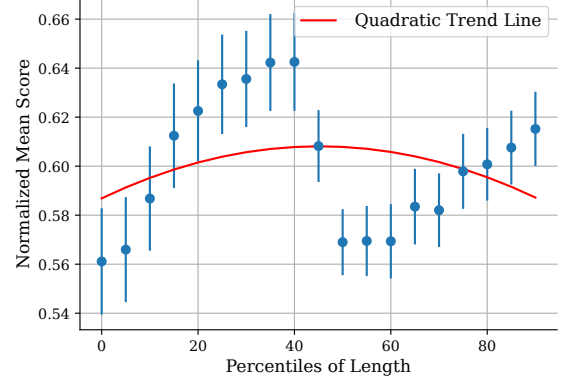


(b) Probability of acceptance by length

Figure 10: Plots for the experiment on ranking suggestions by the probability of acceptance. Histogram (a) shows in which length percentile bin the maximizing suggestion lies and Graph (b) shows the acceptance score by increasing the length of the suggestion. These plots are for $k = 0$ (docstring only)

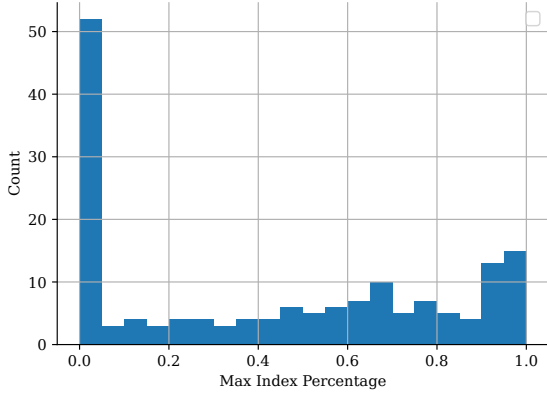


(a) Histogram of position of max suggestion

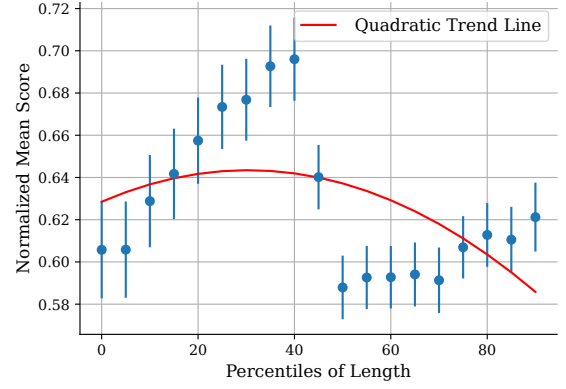


(b) Probability of acceptance by length

Figure 11: Plots for the experiment on ranking suggestions by the probability of acceptance. Histogram (a) shows in which length percentile bin the maximizing suggestion lies and Graph (b) shows the acceptance score by increasing the length of the suggestion. These plots are for $k = 1$ (docstring + first line of solution)

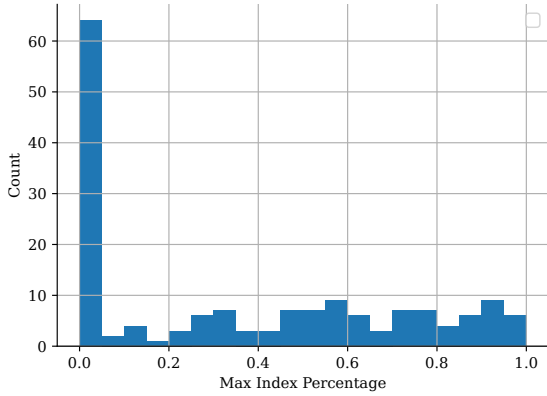


(a) Histogram of position of max suggestion

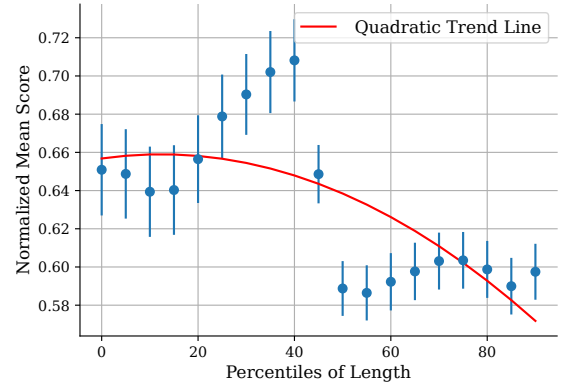


(b) Probability of acceptance by length

Figure 12: Plots for the experiment on ranking suggestions by the probability of acceptance. Histogram (a) shows in which length percentile bin the maximizing suggestion lies and Graph (b) shows the acceptance score by increasing the length of the suggestion. These plots are for $k = 2$ (docstring + first two lines of solution)



(a) Histogram of position of max suggestion



(b) Probability of acceptance by length

Figure 13: Plots for the experiment on ranking suggestions by the probability of acceptance. Histogram (a) shows in which length percentile bin the maximizing suggestion lies and Graph (b) shows the acceptance score by increasing the length of the suggestion. These plots are for $k = 3$ (docstring + first three lines of solution)